

UNIT II - Problem Solving

- **Solving Problems by Searching in Artificial Intelligence (AI)**

Introduction:

In Artificial Intelligence (AI), many problems are solved by searching through possible solutions. Search is a fundamental concept used in areas like robotics, game playing, route planning, and puzzle solving.

Search problems are typically solved by exploring a state space to find a path from an initial state to a goal state.

- **Problem-Solving Agents :**

A problem-solving agent follows these steps:

Goal formulation – Define what needs to be achieved.

Problem formulation – Define states, actions, and goal conditions.

Search – Explore possible states to find a solution.

Execute – Perform the actions.

Components of a Search Problem

A search problem consists of:

Initial State – Starting point.

State Space – All possible states.

Actions (Operators) – Ways to move between states.

Transition Model – Result of applying an action.

Goal Test – Checks if goal is reached.

Path Cost – Cost of a solution path.

Example: Going from Home to School

1. Initial State

Home

(Starting point)

2. State Space

All locations you can travel to:

Home → Park → Shop → School

3. Actions

Walk to a neighboring location:

Home → Park

Park → School

Home → Shop

4. Transition Model

Shows what happens when you take an action:

From Home, walking to Park → Now you are at Park

5. Goal Test

Are you at School? 

6. Path Cost

Can be measured as distance or time:

Home → Park = 5 min

Park → School = 10 min

Total path cost = 15 min

Solution Path:

Home → Park → School (Cost = 15 min)

- **Search Algorithms**

Definition :

A search algorithm is a method or procedure used to explore possible options to find a solution to a problem. In AI, it helps agents or programs find paths, make decisions, or optimize solutions.

Key Components of a Search Problem:

Initial State: Starting point of the search.

Goal State: Desired solution.

Actions/Operators: Possible moves to change states.

Transition Model: How each action changes the state.

Path Cost ($g(n)$): Cost of reaching a state via a path.

State Space: The set of all possible states (can be visualized as a graph).

Example:

Solving an 8-puzzle problem: moving tiles to reach the goal arrangement.

GPS navigation: finding the shortest path between two locations.

Classification:

Uninformed (Blind) Search – no additional info about goal.

Informed (Heuristic) Search – uses knowledge about the problem to make better decisions.

- **Uninformed Search Strategies**

Definition :

Uninformed search explores the search space without any knowledge of goal location, only the structure of the state space.

Types of Uninformed Search:

1. Breadth-First Search (BFS)

Explores all nodes level by level.

Guarantees shortest path if all actions cost equally.

Pros: Complete, optimal (if costs equal)

Cons: High memory usage

Example: Searching all rooms floor by floor in a building.

2. Depth-First Search (DFS)

Explores deepest nodes first.

Can get stuck in loops (needs cycle checking).

Pros: Low memory usage

Cons: Not guaranteed to find shortest path or solution

Example: Exploring one branch of a maze fully before backtracking.

Uniform-Cost Search (UCS)

Expands the node with lowest cumulative cost first.

Uses priority queue for node selection.

Pros: Finds optimal solution, complete

Cons: Can be slow for large spaces

Example: Finding cheapest flight among multiple options.

Iterative Deepening Search (IDS)

Combines DFS's memory efficiency and BFS's completeness.

Repeatedly searches with increasing depth limit.

Pros: Complete, low memory usage

Cons: Repeated exploration of states at shallow depths

- **Informed (Heuristic) Search Strategies**

Definition:

Informed search uses knowledge about the goal (heuristics) to guide the search and reduce time/space complexity.

Key Idea:

Use a heuristic function ($h(n)$) that estimates the cost to reach the goal from node n .

Common Algorithms:

Greedy Best-First Search

Chooses the node that looks closest to the goal.

Evaluation function: $f(n) = h(n)$

Pros: Fast, often finds a solution quickly

Cons: Not guaranteed optimal, can get stuck in loops

A Search*

Combines path cost and heuristic:

$$f(n) = g(n) + h(n)$$

$g(n)$ = cost to reach node n

$h(n)$ = estimated cost to goal

Pros: Complete and optimal (if $h(n)$ is admissible)

Cons: Can be slow if state space is huge

Examples of Heuristics:

8-Puzzle: number of misplaced tiles, Manhattan distance

GPS Navigation: straight-line distance to destination

Chess AI: number of pieces captured, board control

- **Heuristic Functions**

Definition:

A heuristic function is a way to estimate the distance from a current state to the goal.

Properties of a Good Heuristic:

Admissible: Never overestimates cost.

Consistent (Monotonic):

$$h(n) \leq c(n, n') + h(n')$$

Ensures $f(n)$ never decreases along a path.

Design Tips:

Use domain knowledge.

Simple to compute but effective.

Examples:

8-Puzzle → Manhattan distance

Traveling Salesman → straight-line distance between cities

- **Search in Complex Environments**

Definition:

Complex environments are problems with:

Large state space – too many possible states.

Dynamic changes – environment changes over time.

Partial observability – agent doesn't see full state.

Uncertain actions – outcomes may be probabilistic.

Examples:

Robotics: navigating a warehouse with moving obstacles

Autonomous vehicles: dynamic traffic and weather conditions

- **Local Search and Optimization Problems**

Definition:

Local search algorithms focus on single current solution, trying small changes to improve it.

Useful for very large problems where global search is impractical.

Key Algorithms:

1. Hill Climbing:

Moves to neighbor with better value.

Risk: stuck at local maxima/minima.

2. Simulated Annealing:

Sometimes moves to worse solution to escape local maxima.

Inspired by metal cooling process.

3. Genetic Algorithms (GA):

Population-based search.

Uses selection, crossover, mutation to evolve solutions.

Inspired by natural selection.

Optimization Problem Example:

Traveling Salesman Problem (TSP): Find the shortest route visiting all cities.

Job Scheduling: Assign tasks to minimize total completion time.

Quick Summary Table

Topic	Key Idea	Example
Search Algorithm	Step-by-step method to find a solution	Maze, 8-puzzle
1.Uninformed Search	No extra info, explores blindly	BFS, DFS
2.Informed Search	Uses heuristics to guide search	A*, Greedy
3.Heuristic Function	Estimates distance to goal	Tiles out of place
4.Complex	Many states, dynamic changes	Self-driving cars
Environment		
5.Local Search &	Start with solution & improve	Traveling salesman, hill climbing
Optimization		